Deliverable 2 - INF201

Group members: Rishanth Jeevahen. Djordje Pesut GitHub repository: https://github.com/djolepe06-bitWeek44_deliverable2.git

In [1]:
```
import json
import re
from pathlib import Path
import pandas as pd

# Support running from either the project folder or its parent.
data_dirs = (Path("d2_data"), Path("data"), Path("project_2") / "d2_data", Path(
DATA_DIR = next((path for path in data_dirs if path.is_dir()), None)
if DATA_DIR is None:
    raise FileNotFoundError("Could not find d2_data or data. Run from the projec
```

Task 1.1: Robust file reader

In [2]:
```
def read_log_file(file_path: Path) -> tuple[str, str]:
    """Read a UTF-8 or ISO-8859-1 file. Returns (file_text_content, encoding_use
    file_path = Path(file_path)
    try:
        content = file_path.read_text(encoding="utf-8")
        return content, "utf-8"
    except UnicodeDecodeError:
        content = file_path.read_text(encoding="ISO-8859-1")
        return content, "ISO-8859-1"
```

Task 1.2: Log parser with regular expressions

In [3]:
```
LINE_PATTERN = re.compile(
    r"^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]\s+"
    r"\[(?P<register_id>[^\]]+)\]\s+"
    r"item=(?P<item>.+?)\s+"
    r"price=(?P<price>-?\d+(?:\.\d+)?)\s+"
    r"(?P<currency>[A-Z]{3})\s+"
    r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)\s*$",
    re.MULTILINE,
)

def parse_log_file(content: str) -> list[dict]:
    """Return one dictionary per valid sales line; price is converted to float."
    records = []
    for match in LINE_PATTERN.finditer(content):
        record = match.groupdict()
        record["price"] = float(record["price"])
        records.append(record)
    return records
```

Task 2.1: Recursive directory search (no glob / rglob / os.walk)

In [4]:
```python
def find_log_files(directory, extension: str = ".log") -> list[Path]:
    """Recursively find all files with the given extension."""
    directory = Path(directory)
    found: list[Path] = []
    for item in directory.iterdir():
        if item.is_dir():
            found.extend(find_log_files(item, extension))
        elif item.is_file() and item.suffix == extension:
            found.append(item)
    return found
```

Task 2.2: Recursive search for active registers in the JSON config

In [5]:
```python
def get_active_register_ids(node) -> set[str]:
    """Return ids of all registers with "active": true, at any nesting depth."""
    active: set[str] = set()
    if isinstance(node, dict):
        if node.get("active") is True and isinstance(node.get("id"), str):
            active.add(node["id"])
        for value in node.values():
            active |= get_active_register_ids(value)
    elif isinstance(node, list):
        for element in node:
            active |= get_active_register_ids(element)
    return active
```

In [6]:
```python
example = {"company": "Campus Coffee", "regions": [{"region_name": "Main Campus"
    {"name": "Library Cafe", "registers": [{"id": "POS_01", "active": True}, {"i
get_active_register_ids(example)
```

Out[6]:
```
{'POS_01'}
```

Test on the real data

In [7]:
```python
log_files = find_log_files(DATA_DIR)
config = json.loads((DATA_DIR / "store_config.json").read_text(encoding="utf-8")
active_ids = get_active_register_ids(config)

all_records = []
for path in log_files:
    text, encoding = read_log_file(path)
    records = parse_log_file(text)
    all_records.extend(records)
    print(f"{path.relative_to(DATA_DIR)}: {encoding}, {len(records)} valid lines

print("Log files:", len(log_files), "| records:", len(all_records), "| active:",
pd.DataFrame(all_records).head()
```

```
2026\library_cafe\POS_01\register_sales.log: utf-8, 80 valid lines
2026\library_cafe\POS_02\register_sales.log: utf-8, 80 valid lines
2026\library_cafe\POS_03\register_sales.log: utf-8, 80 valid lines
2026\science_building\POS_01\register_sales.log: utf-8, 80 valid lines
2026\science_building\POS_02\register_sales.log: utf-8, 80 valid lines
2026\science_building\POS_03\register_sales.log: utf-8, 80 valid lines
2026\student_union\POS_01\register_sales.log: utf-8, 80 valid lines
2026\student_union\POS_02\register_sales.log: utf-8, 80 valid lines
2026\student_union\POS_03\register_sales.log: utf-8, 80 valid lines
Log files: 9 | records: 720 | active: ['POS_01', 'POS_02', 'POS_03']
```

Out[7]:

	timestamp	register_id	item	price	currency	status
0	2026-10-15 07:00:58	POS_01	Croissant	999.00	USD	SUCCESS
1	2026-10-15 07:02:57	POS_01	Croissant	3.63	NOK	SUCCESS
2	2026-10-15 07:04:08	POS_01	Croissant	3.86	USD	CANCELLED
3	2026-10-15 07:06:02	POS_01	Espresso	2.89	EUR	SUCCESS
4	2026-10-15 07:08:42	POS_01	Sandwich	7.30	EUR	SUCCESS

Task 3: Functions for cleaning and analysing data using NumPy

In [8]:

```python
import numpy as np


def remove_outliers(pricedata: list) -> np.ndarray:
    """Replaces detected outlier values with the median of the valid prices"""

    price = np.array(pricedata, dtype=float).ravel()
    mu = np.mean(price)
    sigma = np.std(price)
    mask = abs(price - mu) > 3 * sigma
    median = np.median(price[~mask])
    price[mask] = median

    return price


def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> n
    """Computes the element-wise refund ratio using NumPy array operations:"""

    epsilon = 10**(-6)
    refund_ratio = refund_count / (sales_count + epsilon)

    return refund_ratio


# Test
cleaned = remove_outliers([r["price"] for r in all_records])
assert cleaned.max() < 999
assert len(cleaned) == len([r["price"] for r in all_records])
print(cleaned)
```

```
[4.05 3.63 3.86 2.89 7.3  4.2  4.52 2.89 4.29 4.3  7.95 4.05 3.25 3.8
 3.32 3.4  3.47 3.8  4.58 4.37 4.05 3.87 7.84 4.18 3.77 4.41 3.66 4.41
 4.68 2.62 3.9  4.66 3.37 3.46 7.49 4.41 3.46 3.76 3.99 4.44 3.81 4.65
 3.43 2.67 4.5  7.03 3.33 4.49 4.38 7.25 7.28 2.51 4.07 4.4  4.19 3.69
 3.15 4.93 3.18 2.92 3.2  7.18 7.25 4.43 3.06 4.72 4.73 7.09 4.97 3.47
 3.59 7.3  2.82 4.44 7.91 7.17 7.07 2.87 3.07 4.49 7.5  3.45 3.51 3.37
 3.89 4.38 7.26 4.28 3.21 3.85 3.92 2.55 4.04 7.22 4.58 4.57 7.46 3.1
 4.21 4.42 3.85 7.07 4.58 7.82 3.44 3.83 7.51 3.75 7.22 7.43 3.   4.05
 3.09 4.42 4.05 7.82 3.03 7.64 7.96 7.58 4.3  4.98 4.15 4.05 3.   7.
 4.53 4.21 3.76 4.88 3.87 7.55 4.65 4.05 3.41 3.43 3.63 7.63 3.62 3.51
 4.05 3.94 4.3  3.2  3.31 4.81 3.04 3.8  7.93 7.93 4.32 7.82 4.8  7.91
 3.05 7.22 3.91 4.55 4.23 7.4  4.66 3.45 3.85 2.53 7.9  3.44 4.45 4.05
 2.63 7.48 7.76 4.67 7.11 3.38 4.06 2.84 3.73 4.66 3.54 2.58 4.57 4.37
 7.57 2.82 3.21 7.48 4.48 4.19 7.25 4.08 3.95 4.55 3.91 4.05 4.29 3.71
 3.63 3.05 3.55 2.79 3.98 4.48 7.67 4.15 3.38 4.05 4.65 3.81 3.23 4.62
 7.01 3.8  3.94 4.78 7.85 3.97 4.87 3.07 3.97 4.39 3.89 3.81 4.12 3.26
 7.84 7.7  2.54 3.06 4.19 4.38 4.38 4.53 4.66 3.28 4.   4.4  7.61 4.25
 7.88 2.89 3.97 3.93 3.93 4.98 3.92 3.62 4.87 4.18 4.45 3.98 4.28 7.68
 3.07 7.45 3.89 4.29 4.27 4.08 3.02 4.06 4.04 7.93 7.03 3.38 4.41 3.76
 3.83 2.97 3.58 3.08 3.7  3.59 3.95 4.46 2.72 4.36 7.56 3.88 4.27 3.55
 4.08 3.46 4.48 4.62 4.22 3.72 4.71 4.94 4.01 4.26 4.66 2.86 3.6  4.53
 4.43 3.88 2.55 4.41 4.29 7.71 7.15 7.71 4.05 4.13 3.93 7.32 4.79 4.06
 4.47 4.01 2.96 3.6  4.24 3.77 2.95 4.09 4.14 3.71 4.01 7.82 3.86 3.93
 3.72 3.07 4.31 3.23 4.99 3.82 4.09 7.53 4.28 4.63 3.42 3.91 7.98 4.05
 3.52 3.48 4.1  4.13 4.61 4.71 3.48 3.98 4.05 3.33 4.05 2.61 7.58 3.01
 4.49 3.05 3.58 4.01 3.25 3.82 7.29 4.51 3.18 7.45 3.85 7.24 3.46 4.05
 3.91 4.24 3.95 3.92 4.24 4.07 3.23 7.77 3.75 4.69 3.29 3.1  4.33 3.87
 4.56 7.65 4.62 3.6  7.2  7.58 4.2  4.95 4.02 4.44 4.22 7.46 3.21 7.57
 3.6  4.1  3.46 3.29 3.68 2.73 3.25 3.82 3.06 7.24 4.03 4.2  2.64 7.76
 3.68 3.9  3.65 3.04 3.55 7.47 3.86 7.32 3.15 3.83 3.51 3.18 4.05 3.47
 7.67 3.28 3.64 4.6  2.52 4.59 3.69 4.43 4.65 4.58 3.94 7.21 3.33 3.6
 4.48 4.38 3.24 2.78 4.66 3.42 7.41 2.68 3.71 3.98 2.55 4.06 2.94 7.49
 3.24 3.73 3.08 4.52 3.06 7.36 7.43 3.39 7.94 4.28 4.33 7.79 7.07 3.73
 2.9  4.16 7.53 4.42 4.44 4.24 4.65 4.55 3.74 7.33 3.82 3.33 3.63 3.19
 4.03 4.15 4.55 3.71 2.54 4.12 3.7  2.92 4.62 7.13 4.07 3.23 3.07 7.3
 2.53 2.83 7.36 4.73 7.06 2.88 4.   3.11 4.22 7.19 7.09 4.74 2.86 3.68
 4.05 4.98 3.46 3.01 7.34 4.   3.46 7.91 4.49 4.67 3.32 3.28 4.43 3.17
 3.31 3.64 4.05 7.49 4.02 4.1  3.69 4.55 2.62 3.27 4.9  4.12 4.87 4.31
 4.08 3.38 4.82 3.6  4.26 2.98 4.79 4.29 2.67 4.38 4.92 4.24 3.3  3.46
 3.17 2.68 4.01 4.92 4.18 4.21 3.77 2.97 3.91 4.75 4.26 7.72 2.58 7.13
 3.21 3.55 4.4  4.05 3.23 3.29 4.09 4.43 7.61 2.98 3.15 4.55 4.77 4.21
 7.8  4.45 7.19 3.57 7.54 4.85 4.36 3.71 3.69 2.84 4.58 3.11 3.5  3.05
 4.85 3.45 4.61 2.79 4.24 4.03 4.24 3.53 7.51 3.76 7.74 4.78 2.68 2.8
 3.67 4.02 3.77 4.19 4.35 3.74 3.44 3.05 3.58 3.71 4.11 2.79 7.22 4.67
 4.23 3.26 4.14 3.51 7.84 2.82 4.32 3.94 4.05 4.11 4.58 3.29 7.14 3.88
 3.13 3.49 4.86 3.76 4.01 3.52 7.3  3.4  3.37 7.3  3.9  3.51 3.69 2.61
 4.62 2.87 4.05 2.52 3.59 4.05 4.03 3.36 4.23 7.31 4.36 4.27 3.45 3.92
 3.19 4.1  3.26 3.36 4.58 3.91 4.67 3.91 3.13 4.87 3.7  4.35 4.53 3.69
 3.16 4.93 3.92 4.17 4.23 4.37 3.46 7.49 3.87 3.92 3.57 4.2  4.87 3.79
 4.18 3.56 4.14 7.08 3.05 2.62 4.05 3.73 3.38 2.88 7.4  4.3  4.01 3.36
 3.1  3.66 7.68 4.02 3.19 7.92 3.08 4.1  3.29 4.11 3.9  4.48 7.24 4.17
 4.61 4.64 3.23 4.09 2.6  7.02]
```

Task 4: Data analysis using Pandas

In [9]:
```python
def convert_list(transaction: list) -> pd.DataFrame:
    "Converts the list of transaction dictionaries read from the log files into
    frame = pd.DataFrame(transaction)
```

```
    if frame.empty:
        return pd.DataFrame(columns=["timestamp", "register_id", "item", "price"


    frame["timestamp"] = pd.to_datetime(frame["timestamp"], errors = "coerce")
    frame["price"] = pd.to_numeric(frame["price"], errors="coerce")

    frame = frame.dropna(subset=["timestamp", "price"]).sort_values("timestamp")
    frame = frame.set_index("timestamp")

    return frame


def group_data(frame: pd.DataFrame) -> sum:

    "Groups data by item and resamples transactions in 1-hour windows (1h), calc

    if frame.empty:
        return pd.DataFrame(columns=["item", "timestamp", "price"])


    df = frame.copy()
    df["price"] = pd.to_numeric(df["price"], errors="coerce")
    df = df.dropna(subset=["price"])

    hourly = (
        df.groupby([pd.Grouper(freq="1h"), "item"])["price"]
        .sum()
        .reset_index()
        .rename(columns={"timestamp": "hour"})
    )
    hourly["hour"] = pd.to_datetime(hourly["hour"])

    return hourly


def pivot_table(frame: pd.DataFrame) -> tuple:
    """Creates a pivot table displaying total transaction -
    revenue grouped by item across status categories (SUCCESS, REFUND, CANCELLED

    if frame.empty:
        return pd.DataFrame(columns=["SUCCESS", "REFUND", "CANCELLED"])

    df = frame.copy()
    df["price"] = pd.to_numeric(df["price"], errors="coerce")
    df = df.dropna(subset=["price"])

    pivot = (
        df.pivot_table(
            index="item",
            columns="status",
            values="price",
            aggfunc="sum",
            fill_value=0
        )
         .reindex(columns=["SUCCESS", "REFUND", "CANCELLED"], fill_value=0)
        )
    return pivot
```

```
def calculate_refund_ratio(frame: pd.DataFrame) -> float:
    "calculates the refund ratio per item"

    if frame.empty:
        return pd.DataFrame(columns=["item", "refund_ratio"])

    df = frame.copy()
    df["status"] = df["status"].astype(str)

    sales_count = df.groupby("item").size()
    refund_count = df[df["status"] == "REFUND"].groupby("item").size()

    all_items = sales_count.index
    refund_count = refund_count.reindex(all_items, fill_value=0)

    ratios = compute_refund_ratio(
        sales_count.to_numpy(dtype=float),
        refund_count.to_numpy(dtype=float)
    )

    result = pd.DataFrame({
        "item": all_items,
        "refund_ratio": ratios
    }).set_index("item")

    return result
```

In [10]:

```
# test
all_records = []

for path in log_files:
    text, encoding = read_log_file(path)
    all_records.extend(parse_log_file(text))

df = convert_list(all_records)
df.head()

hourly = group_data(df)
print(hourly.head())

status_report = pivot_table(df)
print(status_report)

refund_report = calculate_refund_ratio(df)
print(refund_report)
```

```
                 hour        item    price
0 2026-10-15 07:00:00  Cappuccino  2194.84
1 2026-10-15 07:00:00   Croissant  2242.98
2 2026-10-15 07:00:00    Espresso  2149.93
3 2026-10-15 07:00:00       Latte  3234.76
4 2026-10-15 07:00:00      Muffin  2169.69
status      SUCCESS   REFUND  CANCELLED
item
Cappuccino  3389.04    83.26      40.58
Croissant   3294.97    87.70    1045.07
Espresso    3231.74    50.61      69.48
Latte       5375.10    97.07      89.32
Muffin      2240.66  1080.05      45.50
Sandwich     516.46  1200.98     156.39
            refund_ratio
item
Cappuccino      0.160000
Croissant       0.198276
Espresso        0.140496
Latte           0.160305
Muffin          0.220183
Sandwich        0.237288
status      SUCCESS   REFUND  CANCELLED
item
Cappuccino  3389.04    83.26      40.58
Croissant   3294.97    87.70    1045.07
Espresso    3231.74    50.61      69.48
Latte       5375.10    97.07      89.32
Muffin      2240.66  1080.05      45.50
Sandwich     516.46  1200.98     156.39
            refund_ratio
item
Cappuccino      0.160000
Croissant       0.198276
Espresso        0.140496
Latte           0.160305
Muffin          0.220183
Sandwich        0.237288
```

DISCUSSION

Task 5: Create full workflow

In [11]:

```python
def run_pipeline(date_dir: Path = DATA_DIR) -> pd.DataFrame:
    " Full workflow: logs in, four report files out."

    # Finds all logsfiles and extract active register IDs
    log_files = find_log_files(date_dir)
    config = json.loads((date_dir/"store_config.json").read_text(encoding="utf-8
    active_ids = get_active_register_ids(config)

    records = []
    for path in log_files:
        text, encoding = read_log_file(path)
        records.extend(parse_log_file(text))
```

```
    df = convert_list(records)

    # keep only active register IDs
    df = df[df["register_id"].isin(active_ids)]

    # clean outliers per item
    cleaned_prices = []
    for item_name, group in df.groupby("item"):
        cleaned = remove_outliers(group["price"].tolist())
        cleaned_prices.extend(cleaned.tolist())

    # replace prices in same order
    df = df.copy()
    df["price"] = cleaned_prices

    # correct NOK currency error
    df.loc[df["currency"] == "NOK", "price"] = df.loc[df["currency"] == "NOK", "

    # sort by timestamp
    df = df.sort_index()

    # save cleaned dataframe
    df.reset_index().rename(columns={"timestamp": "timestamp"}).to_csv("cleaned_

    # Hourly revenue
    hourly = group_data(df)
    hourly["hour"] = hourly["hour"].dt.strftime("%Y-%m-%d %H:%M:%S")
    hourly.to_json("hourly_revenue.json", orient="records", indent=2)

    # Satus pivot table
    status_table = pivot_table(df)
    status_table.to_csv("status_pivot_table.csv")

    # Refund ratios
    refund_ratio = calculate_refund_ratio(df)
    refund_ratio.reset_index().rename(columns={"index": "item"}).to_csv("item_re

    print("Workflow complete.")
    print("Files generated:")
    for name in ["cleaned_transactions.csv", "hourly_revenue.json", "status_pivo
        print(name, Path(name).exists())

    return df
```

In [12]:

```
# test

# 1) Run the workflow
df = run_pipeline()

# 2) Basic sanity checks
assert isinstance(df, pd.DataFrame)
assert not df.empty
assert df.index.name == "timestamp"
assert "price" in df.columns
assert "currency" in df.columns
assert "status" in df.columns

# 3) Timestamp should be datetime index
assert pd.api.types.is_datetime64_any_dtype(df.index)
```

```
# 4) No absurd price outliers remain
assert df["price"].max() < 200, "Price outlier still present"

# 5) NOK prices should be corrected
nok_prices = df[df["currency"] == "NOK"]["price"]
if not nok_prices.empty:
    assert nok_prices.mean() < 100, "NOK prices still too high"

# 6) Verify output files exist
for f in ["cleaned_transactions.csv", "hourly_revenue.json",
          "status_pivot_table.csv", "item_refund_ratios.csv"]:
    assert Path(f).exists(), f"Missing file: {f}"

print("All checks passed.")

print(df.head())
print(df["price"].describe())
print("Max price:", df["price"].max())
print("NOK mean:", df[df["currency"] == "NOK"]["price"].mean())

for f in ["cleaned_transactions.csv", "hourly_revenue.json",
          "status_pivot_table.csv", "item_refund_ratios.csv"]:
    print(f, Path(f).exists())
```

```
Workflow complete.
Files generated:
cleaned_transactions.csv True
hourly_revenue.json True
status_pivot_table.csv True
item_refund_ratios.csv True
All checks passed.
                    register_id       item  price currency   status
timestamp
2026-10-15 07:00:15      POS_02     Muffin   3.93      USD   REFUND
2026-10-15 07:00:23      POS_03   Espresso   4.40      USD  SUCCESS
2026-10-15 07:00:32      POS_02  Croissant   4.62      USD  SUCCESS
2026-10-15 07:00:35      POS_03  Croissant   3.82      EUR  SUCCESS
2026-10-15 07:00:41      POS_01  Croissant  44.50      NOK  SUCCESS
count    720.000000
mean      18.301319
std       21.648282
min        2.520000
25%        3.770000
50%        4.580000
75%       35.200000
max       79.300000
Name: price, dtype: float64
Max price: 79.3
NOK mean: 45.66111111111111
cleaned_transactions.csv True
hourly_revenue.json True
status_pivot_table.csv True
item_refund_ratios.csv True
```

GenAI Use (Djordje)

Tool: Claude (Anthropic), one long conversation on 29-30 October 2026.

How I used it

- **Task 3 (NumPy):** Asked for hints on why my code seemed to fail. Claude first suggested the 3σ rule might miss the 999 value, then corrected itself: with `np.std` (ddof=0) and 21 values, the z-score can reach about 4.47, so the outlier is caught.
- **Error messages:** Got explanations for `float() argument must be a string or a real number, not 'dict'` (I passed whole records instead of prices) and `KeyError: 'prices'` (the column is called `price`).
- **Task 4 (Pandas):** Claude proposed `convert_list`, `group_data`, `pivot_table` and `calculate_refund_ratio`, and fixed the return types in my draft. The choice of `sales_count` = number of SUCCESS transactions came from this suggestion.
- **Task 5 (workflow):** Claude sketched `run_pipeline`. Key choices: remove outliers before the NOK correction, and filter out inactive registers.
- **Git and GitHub:** Help with `git log`, `git config`, branches and cloning, and with which files to include in the submission.
- **Submission requirements:** Helped build a checklist from the assignment text.

What I did myself

- Wrote Tasks 1, 2 and 3 and tested them on the data.
- Ran and verified the code from Claude against the output (max price, NOK price level, all four output files created).
- Made the decisions on active registers and the refund ratio, and wrote the discussion.

Reflection

- Claude was useful for debugging and explanations, but some answers were imprecise (e.g. the z-score reasoning in Task 3), so everything had to be run and checked.
- I have tried to understand the code well enough to explain it in the video.

GenAI use (Cpilot)

task 4:

1. It helped me to construct the skeleton of every func inn task 4
2. helped my to construct the work flow func for task 5
3. The inf201 folder wasn't a Git repository; it contained several separate repositories.
4. We copied the project files into Week44_deliverable2/project_2.